Homework Assignment 3: Unconstrained Optimization

- **Learning outcome:** By completing this assignment, you will be able to apply the optimality conditions to unconstrained problems by hand, carry out the gradient and Newton methods with exact and inexact (Armijo) line search, explain how conditioning and Hessian definiteness affect their behaviour, and implement and compare these algorithms in code.
- You may choose to use either programming language (MATLAB or Python) for the parts where coding is required. The course notebooks needed for Questions 4 and 5 are linked in those questions; the data files for Question 6 (`Q.txt`, `b.txt`, and `c.txt`) are provided in this GitHub folder.
- Your submission should include: (i) a PDF report named `<McGill_ID>_HW3.pdf` explaining your solution approach, and (ii) either a published MATLAB script or a Python notebook.
- For Questions 4, 5, and 6 you may work in teams, but you must declare whom you worked with and submit your own work (no group submissions).

---

1. **Gradient and Newton methods by hand** (Exercise 4.24, Papalambros & Wilde, 3rd edition)

   Consider the unconstrained minimization problem

   $$\min_{x_1,x_2} \; f(x_1,x_2) = (x_1 - x_2)^2 + (x_2 - 1)^2.$$

   Parts (a)–(c) are to be done by hand; code may be used only for the plot in part (d).

   (a) Find the minimizer $\mathbf{x}^*$ analytically using the first-order necessary conditions, and prove that it is the global minimizer.

   (b) Starting from $\mathbf{x}_0 = (0,0)^T$, perform three iterations of the gradient method with exact line search. Show the step size α_k and the iterate $\mathbf{x}_{k+1}$ at each iteration.

   (c) Starting from the same point, perform Newton's method until convergence.

   (d) Plot the iterates from parts (b) and (c) on a contour plot of f. Compare the two methods and explain the behaviour you observe.

2. **Stationary points** (Exercise 4.26, Papalambros & Wilde, 2nd edition)

   Consider the function

   $$f(x_1,x_2) = x_1 + x_1^{-1} + x_2 + x_2^{-1}.$$

   (a) Find the stationary points of the function.

   (b) Examine whether they are minimizers, maximizers, or saddle points.

3. **Conditioning of a quadratic**

   Consider the minimization problem

$$\min_{x_1,x_2} \; f(x_1,x_2) = \frac{1}{2}\left(x_1^2 + 2\,(1-\varepsilon)\,x_1x_2 + x_2^2\right),$$

   where $\varepsilon \in (0,1]$.

   (a) Find the condition number of the Hessian of f.

   (b) What happens to the condition number as $\varepsilon \to 0$?

4. **Stationary points and Hessian definiteness in code**

   Become familiar with the following course notebooks:

   - `L05_SOSC_demonstration.ipynb`: Section 2 provides a reusable stationary-point finder (`find_stationary_points`) and an SOSC classifier (`classify_stationary_point`).
   - `L06_stabilization_and_scaling.ipynb`: Section 2 applies Newton's method, with and without Hessian modification, to the function

$$f(x_1,x_2) = \frac{1}{3}x_1^3 + x_1x_2 + \frac{1}{2}x_2^2 + 2x_2 - \frac{2}{3}.$$

   MATLAB users may reproduce the same steps in MATLAB.

   (a) Using `find_stationary_points` from the L05 notebook, find the stationary points of f. Use at least five different initial guesses and report which stationary point each initial guess converges to.

   (b) Using `classify_stationary_point` (or by computing the eigenvalues of the Hessian yourself), comment on the positive definiteness of the Hessian at each stationary point found in part (a) and state its nature (minimizer, maximizer, or saddle point). Use your result to explain why plain Newton's method in the L06 notebook can converge to a point that is not a minimizer.

5. **Rosenbrock function: gradient method vs. Newton's method**

   The general Rosenbrock function is given by

$$f(\mathbf{x}) = \sum_{i=1}^{n-1}\left[(1-x_i)^2 + 100\,(x_{i+1}-x_i^2)^2\right]. \tag{1}$$

   Using the L06 course notebooks, minimize its 2-dimensional version ($n=2$) with the following two algorithms:

   - **Gradient method with exact line search.** Start from `exact_line_search_gd` in Section 4 of `L06_gradient_descent_and_line_search.ipynb`. That function assumes a quadratic with a constant Hessian $\mathbf{A}$. Modify it for a general function by evaluating the gradient ∇f_k and Hessian $\mathbf{H}_k$ at every iterate (e.g., with `numerical_gradient` and

`numerical_hessian` from `L06_newton_and_quasi_newton.ipynb`, or with the analytical expressions), and use the exact line-search step from the lecture

$$\alpha_k = \frac{\nabla f_k^{\mathrm{T}} \nabla f_k}{\nabla f_k^{\mathrm{T}} \mathbf{H}_k \nabla f_k}.$$

If $\nabla f_k^{\mathrm{T}} \mathbf{H}_k \nabla f_k \leq 0$, this step is not valid; fall back to `armijo_backtrack` for that iteration.

- **Newton's method.** Use `L06_newton_and_quasi_newton.ipynb` `newton_method` from Section 1 of the notebook.

MATLAB users may reproduce the same algorithms in MATLAB.

Run each algorithm from all five of the following initial guesses $\mathbf{x}_0^T$:

$$[-1.2 \;\; 1]; \quad [-1 \;\; -1]; \quad [0 \;\; 0]; \quad [1 \;\; 0]; \quad [1 \;\; 1.2].$$

Terminate when $\|\nabla f(\mathbf{x}_k)\| \leq \epsilon_{\text{tol}}$ with $\epsilon_{\text{tol}} = 0.001$, or when the maximum number of iterations $k_{max} = 5000$ is reached.

(a) For each algorithm and initial guess, report the final solution, the value of the termination criterion, and the number of iterations.

(b) Provide a contour plot that shows all iterates (including the initial guess) generated by each algorithm.

6. **Your own gradient descent with Armijo line search**

Consider the multi-dimensional minimization problem

$$\min_{\mathbf{x}\in\mathbb{R}^n} \; f(\mathbf{x}) = \mathbf{x}^{\mathrm{T}}\mathbf{Q}\mathbf{x} + \mathbf{b}^{\mathrm{T}}\mathbf{x} + c, \tag{2}$$

where $\mathbf{Q}$ is a positive definite matrix. For this problem you need to implement your own gradient descent algorithm, with search direction $\mathbf{d}_k = -\nabla f(\mathbf{x}_k)$ and the following inexact line search based on Armijo's rule:

(1) Pick $\alpha > 0.5$, and choose $\varepsilon \in [0.1, 0.2]$ and $\beta \in [0.8, 0.9]$.

(2) If $f(\mathbf{x}_k + \alpha\mathbf{d}_k) \leq f(\mathbf{x}_k) + \varepsilon\alpha\nabla^{\mathrm{T}} f(\mathbf{x}_k)\mathbf{d}_k$, go to step (4).

(3) Otherwise, set $\alpha \leftarrow \beta\alpha$ and go to step (2).

(4) Stop and accept $\alpha_k = \alpha$.

Your algorithm must terminate when $\|\nabla f(\mathbf{x}_k)\| \leq \epsilon_{\text{tol}}$ or when the maximum number of iterations $k_{\max}$ is reached. Load the data files `Q.txt`, `b.txt`, and `c.txt` (available here) with `np.loadtxt` (Python) or `dlmread` (MATLAB).

(a) Using your **own** implementation (not the course notebooks), solve the problem in Equation (2) and report:

(i) the termination point $\mathbf{x}_k$;

(ii) the objective value at the termination point, $f(\mathbf{x}_k)$;

(iii) the tolerance reached, $\|\nabla f(\mathbf{x}_k)\|$;

(iv) the number of iterations k.

(b) Solve the problem in Equation (2) using the first-order necessary conditions (FONCs). Report $\mathbf{x}^*$ and $f(\mathbf{x}^*)$.

(c) Compare your answers in parts (a) and (b) with those of an available optimization solver: `scipy.optimize.minimize` (Python) or `fminunc` (MATLAB).

About MATLAB

Use the `help` option in MATLAB to find out about available MATLAB functions. The `help fminunc` command will give information on the fminunc function and what its inputs and outputs are.

About Python

Use the `help` option in Python to find out about available Python functions. The following command will give information on the minimize function and what its inputs and outputs are:
`help scipy.optimize.minimize`